

Stacktrace from exception

Changes from previous revision are in blue, wording additions are in green.

I. Introduction and Motivation

Quite often Standard Library exceptions do not provide enough information to diagnose the error. Consider the example:

```
#include <iostream>
#include <stdexcept>
#include <string_view>

void foo(std::string_view key);
void bar(std::string_view key);

int main() {
    try {
        foo("test1");
        bar("test2");
    } catch (const std::exception& exc) {
        std::cerr << "Caught exception: " << exc.what() << '\n';
    }
}
```

The output of the above sample may be the following:

```
Caught exception: map::at
```

That output is quite useless because it does not help the developer to understand in what function the error happened.

This paper proposes to add an ability to get stacktrace of a caught exception, namely to add static method `std::stacktrace::from_current_exception()`:

```
#include <iostream>
#include <stdexcept>
#include <string_view>
#include <stacktrace> // <---

void foo(std::string_view key);
void bar(std::string_view key);

int main() {
    try {
        foo("test1");
        bar("test2");
    } catch (const std::exception& exc) {
        std::stacktrace trace = std::stacktrace::from_current_exception(); // <---
        std::cerr << "Caught exception: " << exc.what() << ", trace:\n" << trace;
    }
}
```

The output of the above sample may be the following:

```
Caught exception: map::at, trace:
0# get_data_from_config(std::string_view) at /home/axolm/basic.cpp:600
1# bar(std::string_view) at /home/axolm/basic.cpp:6
2# main at /home/axolm/basic.cpp:17
```

That output is quite useful! Without a debugger we can locate the source file and the function that has thrown the exception.

More production log examples where stacktraces would help to diagnose the problem faster:

- Serializing 'UserData' failed: bad optional access — a trace would show that the exception is from NormalizePhone function; further logs analysis will show that the phone is missing.
- ERROR [/order-search] exception in 'handler-search-cab-orders' handler in handle_request: _Map_base::at (std::out_of_range) — a trace would show that the exception is from GetOrganization function; reading the code would show that the function has issues with concurrent updates
- Failed to process 'POST' request to '/v1/order-a-cab': bad any_cast — a trace would show that the exception is from authorization function; wrong context is passed to the authorization function.
- Fatal: Missing a colon after a name of object member. — a trace would show that the exception is from a function that parses JSON dictionary of configuration variables.

Here's another motivation example. It allows the developer to diagnose `std::terminate` calls because of throws in `noexcept` function:

```
#include <iostream>
#include <stacktrace>
#include <stdexcept>
#include <unordered_map>

void broken_function() noexcept {
    std::unordered_map<std::string, int> m;
    [[maybe_unused]] auto value = m.at("non-existing-key");
}

int main() {
    std::set_terminate([] {
        auto trace = std::stacktrace::from_current_exception();
        if (trace) {
            std::cerr << "Terminate was called with an active exception:\n"
                << trace << std::endl;
        }
    });

    broken_function();
}
```

Output:

```
Exception trace:
0# std::__throw_out_of_range(char const*) at /build/gcc/src/gcc/libstdc++-v3/src/c++11/functexcept.cc:82
1# std::__detail::_Map_base<std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator ...
2# std::unordered_map<std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > ...
3# broken_function() at /home/axolm/terminate.cpp:8
4# main at /home/axolm/terminate.cpp:17
```

Disclaimer: may not produce traces on all platforms

Finally, an ability to extract stacktraces from exceptions addresses feature requests from "2021 Annual C++ Developer Survey". In it some people were complaining that in their projects exceptions are restricted due to lack of stacktraces.

II. Implementation and ABI stability

Majority of the popular platforms have special functions to allocate/throw/deallocate exceptions: `__cxa_allocate_exception`, `__cxa_throw`, `__CxxThrowException`... To embed a trace into an exception an underlying implementation may be changed, to gather the trace at the point where the exception is thrown.

This can be done without breaking the ABI. Here's a prototype [libsfe](#) that replaces those functions and adds a stacktrace into each exception. Application can switch between exceptions with traces and without via adding and removing a `LD_PRELOAD=/usr/local/lib/libsfe_preload.so`.

To implement the functionality on Windows platform a new version of the `__CxxThrowException` function may be provided by the compiler runtime and to represent an exception with trace a new `dwExceptionCode` code may be reserved for the `RaiseException` [[MSVCExceptions](#)]:

```
[[noreturn]] void __stdcall __CxxThrowException(void* pObj, _ThrowInfo* pInfo) {
    if (std::this_thread::get_capture_stacktraces_at_throw()) {
        struct ParamsNew { unsigned int magic; void* object, _ThrowInfo* info, std::stacktrace* trace; };
        std::stacktrace trace = std::stacktrace::current();
        ParamsNew throwParams = { ?????, pObj, pInfo, &trace };
        RaiseException(0xE06D7364 /*!new dwExceptionCode code!*/, 1, 4, (const ULONG_PTR*)&throwParams);
    }

    struct ParamsOld { unsigned int magic; void* object, _ThrowInfo* info };
    ParamsOld throwParams = { ?????, pObj, pInfo };
    RaiseException(0xE06D7363, 1, 3, (const ULONG_PTR*)&throwParams);
}
```

}

P2490R0 shows more implementation techniques for getting the stacktraces from exceptions without an ABI break.

III. Use cases

The table bellow shows std::stacktrace related techniques to use in different cases

Case	HowTo	Proposal
Need a way to get diagnostic info for an unrecoverable errors. For example for failed assert checks	Just log the trace and abort: std::cerr << std::stacktrace::current(); std::abort();	P0881
Need to get stacktraces from some code paths and you have control over the throw site.	Manually embed the stacktrace into the exception: template <class Base> struct TracedException: Base, std::stacktrace { TracedException(): Base(), std::stacktrace(std::stacktrace::current()) {} }; // ... throw TracedException<std::bad_any_cast>(); Extract it at the catch site: catch (const std::exception& e) { if (auto ptr = dynamic_cast<const std::stacktrace*>(&e); ptr) { std::cerr << *ptr; } // ... }	P0881
Need a trace from the throw site to simplify diagnoses of failed tests.	Use stacktrace::from_current_exception in your testing macro to get the trace: EXPECT_NO_THROW(search_engine_impl("42"));	This proposal
Need a way to diagnose a running application in production.	Turn on stacktrace::from_current_exception in your error handling logic: try { std::this_thread::set_capture_stacktraces_at_throw(trace_exceptions_var); process_request(request); } catch (const std::exception& e) { if (auto trace = std::stacktrace::from_current_exception(); trace) LOG_PROD_DEBUG() << e.what() << " at " << trace; } }	This proposal

To sum up: you need functionality from this paper when you have no control over the throw site or changing all the throw sites is time consuming.

IV. P2490 "Zero-overhead exception stacktraces"

P2490 proposes to capture the stacktrace at entering the catch block rather than at the throw side. In our opinion the approach is interesting, but some of the P2490 preferred design choices have disadvantages.

1. Disadvantages

A. No way to disable stacktrace capturing at runtime in some of the proposed syntaxes

The P2490 proposal does not provide a runtime way to disable capturing stacktraces at entering the catch block in "2.1. Special function":

```
std::atomic<bool> g_debug{false};  
// ...  
try {  
    process_request(request);  
} catch (const std::exception& e) {  
    some_function();
```

```

    if (g_debug) {
        std::cerr << e.what() << " at " << std::stacktrace::from_current_exception();
    }
}

```

In the above sample there is no compile time information to understand if `std::stacktrace::from_current_exception()` is invoked at runtime or not. So the stacktrace is always captured and stored, even if it is not used.

This proposal P2370 provides a way to disable captures and it does not add any noticeable overhead: [online benchmark](#), reports 4087 ticks for `throw+catch` vs 4068 ticks for `throw+catch` with thread local variable modification. The difference is totally lost in noise even for the case with no destructor invocations for local variables and for 0-1 unwinding depth.

B. Stacktraces are lost at `throw`;

```

void process(std::string_view request) { // third party library code
    try {
        auto parsed = ParseRequest(request); // may throw SomeException
        Authorize(parsed); // may throw AuthException
        auto response = MakeResponse(parsed); // may throw SomeException
        Send(response); // may throw SomeException
    } catch (...) {
        CloseConnection();
        throw;
    }
}

// Users code:

try {
    process(request);
} catch (const SomeException& ) {
    std::cerr << std::stacktrace::from_current_exception();
} catch (const AuthException& ) {
}

```

According to the P2490 the rethrown `SomeException` would give stacktrace starting from `process()` which is probably not what the user wished for. There is no way to get the whole trace without changing the code of the `process()` function which may be a third party code that the user has no control of.

Moreover, `std::rethrow_exception` produces stacktrace from the rethrow point, stacktraces from `std::exception_ptr` lose the most important part of the trace.

For a debugging facility that tends to be usable without code modifications such behavior is not the best default.

C. Does not work with zero overhead exceptions

Zero overhead exceptions are close to returning `std::expected`. They reuse the return paths, so when an exception reaches the `catch` block there is no stacktrace left, all the frames were cleaned up. This proposal P2370 may work with zero overhead exceptions if the stacktrace is stored within `std::expected` at throw.

2. Middle ground with P2490

To deal with disadvantage "A" `this_thread::get_capture_stacktraces_at_throw()` is still required. P2490 is against that, however as was shown above the performance impact of such decision is unnoticeable.

To deal with disadvantage "B" use the P2490 "mechanisms to alleviate" by default. If there is a `throw`; in the `catch` block and `this_thread::get_capture_stacktraces_at_throw()` returns `true` then the stacktrace for exception is captured and its frames appended to some storage associated with the exception.

With the above changes in mind revision 2 of this proposal does a minor wording tweaking to make P2490 based implementations conform to the wording of this proposal.

As a result, where to capture the traces (at `throw` exception or at `catch` block) is now a Quality of Implementation issue. Implementations that prefer to capture stacktraces at the `catch` block conform to the wording and still may have the following optimizations:

```

try {
    throw std::runtime_error("sample");
} catch (const std::exception& e) {
    // do nothing
}

```

In the above sample there is no `throw`; or `from_current_exception()`. The code could be optimized to totally skip the `this_thread::get_capture_stacktraces_at_throw()` checks and stacktrace captures.

```
try {
    throw std::runtime_error("sample");
} catch (const std::exception& e) {
    std::cerr << std::stacktrace::from_current_exception();
}
```

In the above sample there is a potential use of `from_current_exception()`. If a `this_thread::get_capture_stacktraces_at_throw()` call returns `false` then the stacktrace capturing is skipped.

```
try {
    throw std::runtime_error("sample");
} catch (const std::exception& e) {
    throw;
}
```

In the above sample there is a potential use of `throw`; if a `this_thread::get_capture_stacktraces_at_throw()` call returns `false` then the stacktrace capturing is skipped.

V. Design decisions

A. Recommend to turn off by default

As was noted in mailing list discussion the proposed functionality increases memory consumption by exceptions. Moreover, it adds noticeable overhead on some platforms.

Because of that we recommend to disable the functionality by default in release builds and enable in debug. To enable the stacktrace capturing on exception object construction for the current thread user would have to use `std::this_thread::set_capture_stacktraces_at_throw(bool enable) noexcept`;

B. Safety

Proposed `std::stacktrace::from_current_exception()` function is a diagnostic facility. It would be used in error processing and other places that are rarely tested. Because of that it follows the `std::stacktrace` design: do not report missing traces as errors, do not throw and treat all the internal errors as a missing trace.

C. Link time flag from `std::stacktrace`

"A Proposal to add stacktrace library" [P0881](#) encouraged implementations to provide a link time flag to disable or enable traces. With disabled tracing `std::stacktrace::from_current_exception()` may be called at runtime and it returns default constructed `std::stacktrace`.

D. Per thread ability to enable

Consider some server under heavy load with 5 different executors. Something goes wrong with one of the executors. Capturing of stacktraces on exception object construction and symbolizing them for all the executors could slow down the server significantly while only traces for 1 executor are needed. So the option should be thread specific.

Such approach scales well to coroutines. Call the `std::this_thread::set_capture_stacktraces_at_throw(bool enable) noexcept`; after context switch and you have per coroutine ability to enable or disable stacktrace capturing at the exception object construction.

E. Allow stacktrace capturing at the catch block

P2490 shows that there may be different techniques to capture the trace. The wording tends to allow different implementations, and because of that calling the `set_capture_stacktraces_at_throw` during stack unwinding may produce empty stacktraces.

VI. Wording

Add to the [stacktrace.syn]:

```
// [stacktrace.basic], class template basic_stacktrace
template<class Allocator>
class basic_stacktrace;

namespace this_thread {
    void set_capture_stacktraces_at_throw(bool enable = true) noexcept;
    bool get_capture_stacktraces_at_throw() noexcept;
}

// basic_stacktrace typedef names
using stacktrace = basic_stacktrace<allocator<stacktrace_entry>;>;
```

Add to the [stacktrace.basic.overview]:

```
static basic_stacktrace current(size_type skip, size_type max_depth,
                               const allocator_type& alloc = allocator_type()) noexcept;
```

```
static basic_stacktrace from_current_exception(
    const allocator_type& alloc = allocator_type()) noexcept;
```

```
basic_stacktrace() noexcept(is_nothrow_default_constructible_v<allocator_type>);
```

Add to the [stacktrace.basic.ctor] after the description of `current()` functions:

```
static basic_stacktrace from_current_exception(const allocator_type& alloc = allocator_type()) noexcept;
```

Effects: Returns a `basic_stacktrace` object with `frames_` containing a stacktrace as if captured at the point where the currently handled exception was thrown by its initial `throw-expression` (i.e. not a `rethrow`), or an empty `basic_stacktrace` object if:

- the initialization of `frames_` failed, or
- stacktrace captures are not enabled for the throwing thread, or
- stacktrace captures for the throwing thread were changed during stack unwinding or exception handling, or
- no exception is being handled, or
- due to implementation-defined reasons.

`alloc` is passed to the constructor of the `frames_` object.

Rethrowing an exception using a `throw-expression` with no operand does not alter the captured stacktrace.

Add section [stacktrace.thread.this] before the [stacktrace.basic.mod] section:

Namespace `this_thread`

[stacktrace.thread.this]

```
void this_thread::set_capture_stacktraces_at_throw(bool enable = true) noexcept;
```

Effects: Invoking the function with the `enable` parameter equal to `true` enables capturing of stacktraces by the current thread of execution at exception object construction, disables otherwise. It is implementation-defined whether the capturing of stacktraces by the current thread of execution is enabled if `set_capture_stacktraces_at_throw` has not been invoked in the current thread of execution.

```
bool this_thread::get_capture_stacktraces_at_throw() noexcept;
```

Returns: whether the capturing of stacktraces by the current thread of execution is enabled and `from_current_exception` may return a non empty `basic_stacktrace`.

Adjust value of the `__cpp_lib_stacktrace` macro in [version.syn] to the date of adoption.

VII. Revision History

Revision 2:

- Analysis of P2490.
- Tweaked the wording to allow P2490 implementation.

Revision 1:

- Renamed `capture_stacktraces_at_throw` into `set_capture_stacktraces_at_throw`, added `get_capture_stacktraces_at_throw` function.
- Consistent usage of "exception object" and "capturing of stacktraces".
- Added an implementation idea for Windows.
- Minor wording clarifications, feature test macro notes and cleanups.

Revision 0:

- Initial proposal.

VIII. Acknowledgements

Many thanks to all the people who helped with the paper! Special thanks to Jens Maurer and Ville Voutilainen for numerous comments and suggestions.

IX. References

[libsfe] Proof of concept implementation. Available online at <https://github.com/axolm/libsfe>.

[MSVCExceptions] "[Compiler Internals - How Try/Catch/Throw are Interpreted by the Microsoft Compiler](#)" by Yanick Salzmänn.

[P2490R0] [Zero-overhead exception stacktraces](#) by Ed Catmur.

[ThrowCatchBenchmark] [Measure execution time of throw+catch without and with modification of a thread local variable.](#)